

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання лабораторної роботи №6
з курсу «Хмарні технології та великі дані»
на тему:
«Fan-In (Мультиплексор)»

Виконав:
студент групи ФеІ-42
Трофимов Володимир
Перевірив:
ас. Керод Т. І.

Зміст

Тема.....	3
Мета	3
Вхідні дані	3
Завдання.....	3
Критерії виконання роботи.....	4
Теоретичні відомості	4
Структура програмної реалізації	5
Тестування програмної реалізації.....	5
Висновок	7
Код програми	8

Тема

Асинхронне неблокуюче мультиплексування декількох джерел даних у єдиний потік за допомогою патерну Fan-In (Мультиплексор).

Мета

Дослідження механізмів конкурентного злиття незалежних асинхронних потоків даних. Розробка типізованого неблокуючого генератора мовою TypeScript за патерном Fan-In (Мультиплексор) із використанням протоколу AsyncIterable, збереженням відносного порядку елементів у межах кожного джерела, ізоляцією помилок через структуру Result<T, E> та коректним вивільненням ресурсів вхідних ітераторів.

Вхідні дані

Параметр	Значення	Призначення
inputs	Масив Stream<T>[] (AsyncIterable<T>[])	Набір незалежних вхідних асинхронних потоків для злиття.
iterator	AsyncIterator<T>	Асинхронний ітератор конкретного потоку для читання наступного чанка (iterator.next())..
index	Ціле невід'ємне число (number)	Індекс джерела в масиві для ідентифікації потоку, що виконав емісію значення
Result<T, E>	{ ok: true, value: T } { ok: false, error: E }	Дискримінантне об'єднання для безпечної передачі результату або винятку без падіння загального конвеєра.

Завдання

- Розробити типізовану асинхронну функцію-генератор fanIn<T>(inputs: Stream<T>[]): AsyncGenerator<T, unknown void,> мовою TypeScript.
- Забезпечити неблокуюче конкурентне опитування джерел через динамічний пул активних промісів за допомогою Promise.race().
- Реалізувати безпечну ізоляцію збоїв: перехоплення винятків окремого ітератора за допомогою допоміжної функції readNext та обгортання їх у типізований результат Result<T, E> без зупинки обробки інших потоків.
- Гарантувати своєчасне завершення вихідного стріму (pending.size === 0) суворо після того, як вичерпано всі активні вхідні потоки.
- Забезпечити гарантоване очищення та закриття відкритих ітераторів через блок finally (iterator.return()) для запобігання витокам пам'яті.

- Створити автономний демонстраційний набір тестів (fanIn-demo.ts) для перевірки злиття 2-х та 3-х потоків, перевірки коректного завершення та ізоляції критичних збоїв.

Критерії виконання роботи

- *Неблокуюче злиття*: надходження нового значення у будь-який вхідний потік негайно передається споживачеві (yield) без очікування повільніших або заблокованих потоків.
- *Збереження локального порядку*: послідовність емісій у межах одного окремого потоку A залишається суворо незмінною ($A_1 \rightarrow A_2 \rightarrow A_3$), хоча порядок чергування з елементами інших потоків B чи C визначається реальною часовою шкалою надходження подій.
- *Коректне закриття конвеєра*: вихідний потік не завершується передчасно, допоки хоча б один вхідний потік продовжує передавати дані, і автоматично закривається після завершення всіх входів.
- *Ізоляція помилок*: виникнення винятку в одному з потоків інкапсулюється в об'єкт { ok: false, error }, не викликає аварійного обриву інших конкурентних потоків та дозволяє їм дочитати всі залишкові дані.
- *Очищення ресурсів*: використання блоку finally гарантує виклик .return() для всіх активних ітераторів за будь-якого сценарію виходу.

Теоретичні відомості

Патерн Fan-In (Мультиплексор) є фундаментальним архітектурним патерном у реактивному програмуванні та системах обробки потоків великих даних (Stream Processing). Його завдання полягає у консолідації багатьох незалежних каналів або джерел подій у єдиний уніфікований вихідний потік даних.

У середовищі JavaScript/TypeScript модель виконання базується на однопоточковому циклі подій (Event Loop). Для обробки дискретних чанків інформації застосовується протокол асинхронної ітерації (Symbol.asyncIterator, AsyncIterable<T>). Традиційні підходи на кшталт послідовного for await...of не підходять для паралельних джерел, оскільки блокують зчитування з наступного потоку, допоки попередній повністю не завершиться або не поверне значення.

Для забезпечення справжньої конкурентності та неблокуючого опитування використовується пул очікування на основі Promise.race(). Для кожного з N вхідних джерел формується проміс на виклик iterator.next(). Метод Promise.race() резолвить той проміс, який першим надав дані, після чого генератор робить yield значення, а для звільненого джерела негайно ініціюється наступний iterator.next().

Особливе місце посідає політика обробки помилок (Error Handling). За замовчуванням необроблений throw в асинхронному ітераторі відхиляє весь спільний проміс, що призводить до втрати даних з інших здорових каналів.

Застосування патерну `Result<T, E>` інкапсулює аварійний стан конкретного джерела у звичайне значення потоку, зберігаючи працездатність решти системи.

Структура програмної реалізації

Програмне рішення організоване у вигляді модульної TypeScript-архітектури::

- `src/fanIn.ts` — ядро патерну мультиплексування:
 - Інтерфейси `Result<T, E>` та `Stream<T>`.
 - Допоміжна асинхронна функція `readNext`, що перехоплює винятки й індексує результати.
 - Основна функція-генератор `fanIn<T>`, що реалізує конкурентний цикл на базі `Map<number, PendingEntry<T>>` та `Promise.race()`, а також блок гарантованого звільнення ресурсів `Promise.allSettled(iterators.map(...))`.
- `fanIn-demo.ts` — автоматизований демонстраційний сценарій, що імітує часові затримки, генерацію помилок та виводить діаграми чергування подій у консоль.

Тестування програмної реалізації

Верифікація коректності реалізації виконана за допомогою демонстраційного середовища `fanIn-demo.ts`, що охоплює 4 контрольні сценарії.

Запуск тестового скрипта виконується командою `npm run demo`.

```
Starting Fan-In demo...

=== Test 1: Two streams ===
A:  --A1----A2----A3
B:   B1----B2----B3
OUT: B1-A1-B2-B3-A2-A3
Fan-In completed for two streams.

=== Test 2: Three streams ===
A:  --A1----A2----A3
B:   B1----B2----B3
C:   C1----C2----C3
OUT: C1-B1-A1-B2-C2-B3-A2-C3-A3
Fan-In completed for three streams.

=== Test 3: Completion behavior ===
A:  --A1----A2
B:   B1----B2----B3----B4
C:   C1
OUT: B1-A1-B2-A2-B3-C1-B4
All inputs completed; fan-in closed.

=== Test 4: Error isolation ===
A:  A1----A2----ERROR
B:   B1----B2----B3----B4
OUT: A1-B1-A2-B2-ERROR(stream A failed)-B3-B4
Fan-In finished after all streams ended, despite the error in A.
```

Рис.1 – Автономний консольний запуск патерну *fan-in*

Аналіз результатів виконання:

- *Test 1 (Злиття двох потоків):* Потік *B* згенерував значення *B1* першим за рахунок меншої стартової затримки (10 мс проти 50 мс у *A*). Далі елементи чергувалися в порядку реальної готовності (*B1-A1-B2-B3-A2-A3*). Локальний порядок $A_1 \rightarrow A_2 \rightarrow A_3$ та $B_1 \rightarrow B_2 \rightarrow B_3$ повністю збережений.
- *Test 2 (Злиття трьох потоків):* Успішно об'єднано три конкурентні канали з різною динамікою затримок (OUT: *C1-B1-A1-B2-C2-B3-A2-C3-A3*). Мультиплексор однаково стабільно працює для довільної кількості *N* джерел.
- *Test 3 (Поведінка при завершенні):* Потік *A* завершився після 2 елементів, потік *C* містив лише 1 елемент із великою паузою (200 мс), а *B* генерував 4 елементи. Мультиплексор продовжував працювати до повного вичерпання останнього активного джерела (*B4*), після чого коректно закрив вихідний генератор.
- *Test 4 (Ізоляція збоїв):* У потоці *A* після значення *A2* було згенеровано виняток (`throw new Error('stream A failed')`). Механізм `readNext` перехопив помилку й обгорнув її в дискримінантний об'єкт `{ ok: false, error }`. Завдяки цьому потік *B* не перервався і безперешкодно передав значення *B3* та *B4*.

Висновок

У ході виконання лабораторної роботи було досліджено теоретичні засади та практичні аспекти проектування реактивних конвеєрів за патерном Fan-In (Мультиплексор) у середовищі TypeScript.

Розроблено високопродуктивну, типізовану реалізацію функції `fanIn`, яка транслює довільну кількість вхідних потоків `AsyncIterable<T>` у єдиний асинхронний вихідний генератор. Завдяки динамічному пулу незавершених промісів та алгоритму вибору найшвидшої відповіді через `Promise.race()`, досягнуто повністю неблокуючу поведінку: жоден швидкий потік не простоює в очікуванні повільного сусіда. Застосування патерну `Result<T, E>` дозволило досягти повної ізоляції помилок — аварійне завершення одного з джерел фіксується як статусний результат і не руйнує передачу даних по інших активних каналах. Очищення ресурсів за допомогою викликів `.return()` у секції `finally` гарантує відсутність «висячих» дескрипторів та витоків пам'яті.

Код програми

Код до даної лабораторної роботи розміщений у репозиторії https://github.com/MrCatman137/Cloud_Technologies у директорії Lab_6 та має повну інструкцію запуску даної програми.